

Napredne baze podataka

Skripta za usmeni ispit

ljetni semestar 2025./2026.

Skripta pokriva sva pitanja koja je profesor tijekom nastave označio kao ispitna, ali organizirana **tematski i s objašnjenjima** umjesto kao popis pitanja za štrebanje. Cilj je razumjeti *zašto*, a ne samo reproducirati definicije.

Zlatnom oznakom **[ispitno pitanje]** označena su pitanja koja je profesor izrijeком najavio. Zelene kutije (**Za usmeni**) sadrže dodatni kontekst i mostove između tema; crvene (**Zamka**) upozoravaju na klasične pogreške i pitanja u koja profesori vole *ubosti*.

Sadržaj

1	Relacijski temelji: JOIN i normalizacija	2
1.1	JOIN operacije	2
1.2	Normalizacija	3
2	Pravila integriteta i indeksi	3
2.1	Referencijski integritet i ograničenja	3
2.2	Indeksi	4
2.3	B-stablo	5
3	Programabilnost baze: funkcije, procedure i okidači	5
3.1	Funkcije vs. pohranjene procedure	5
3.2	DDL nad procedurama: CREATE / ALTER / DROP	6
3.3	Kontrola toka i višestruki rezultati	6
3.4	Iznimke (exceptions)	6
3.5	Okidači (triggers)	6
4	Napredne teme relacijskih baza	8
4.1	Složeni tipovi i polustrukturirani podaci	8
4.2	Vremenski i bitemporalni podaci	8
4.3	Distribuirane vs. centralizirane baze	9
4.4	Transakcije i 2PC	9
4.5	Visoka dostupnost, klasteriranje, failover	10
4.6	Skalabilnost i backup	10
4.7	Zašto relacijske baze	10
5	NoSQL: temelji, vrste i modeli	10
5.1	Što je NoSQL i koje su karakteristike važne	11
5.2	Vrste NoSQL baza	11
5.3	Konzistentnost: relacijske vs. nerelacijske	11
5.4	Agregatni model	12
5.5	Shema podataka i schemaless	12
5.6	Zašto NoSQL i za što	12
5.7	CAP teorem	13
5.8	ACID vs. BASE	13
5.9	Key-value i MongoDB (detaljnije)	14
6	Grafovske baze i Cypher	14
7	Map/Reduce i Aggregation Pipeline Framework	14
7.1	Ideja i faze	15
7.2	Primjer: analiza košarice	15
7.3	Combiner, combinable reducer, finalize	15
8	Polyglot persistence	16
9	NoSQL u RDBMS-ovima i NewSQL	16
10	Brzo ponavljanje pred usmenim	17

1. Relacijski temelji: JOIN i normalizacija

1.1 JOIN operacije

JOIN spaja retke iz dviju (ili više) tablica na temelju uvjeta spajanja (najčešće jednakosti stranog i primarnog ključa). Ključna razlika među tipovima je *što se događa s retcima koji nemaju para u drugoj tablici*.

Definicija: INNER vs. LEFT JOIN

[ispitno pitanje]

INNER JOIN vraća *samo* one retke koji imaju podudaranje u *obje* tablice. **LEFT (OUTER) JOIN** uvijek vraća *sve* retke iz lijeve tablice; ako za neki lijevi redak nema para u desnoj, stupci desne tablice popune se vrijednošću NULL.

```
-- INNER: samo studenti koji imaju barem jedan upisani kolegij
SELECT s.ime, k.naziv
FROM student s
INNER JOIN upis u ON u.student_id = s.id
INNER JOIN kolegij k ON k.id = u.kolegij_id;

-- LEFT: SVI studenti, i oni bez ijednog upisa (k.naziv = NULL)
SELECT s.ime, k.naziv
FROM student s
LEFT JOIN upis u ON u.student_id = s.id
LEFT JOIN kolegij k ON k.id = u.kolegij_id;
```

Koji vraća više redaka? [ispitno pitanje] Na istim tablicama i istom uvjetu **LEFT JOIN** uvijek vraća $\geq$ **redaka** nego **INNER JOIN**. Razlog je izravno u definiciji: **LEFT** sadrži cijeli rezultat **INNER**-a *plus* sve lijeve retke bez para (nadopunjene NULL-ovima). Jednakost nastupa samo ako svaki lijevi redak ima par — tada je skup „siročadi” prazan.

Za usmeni: cijela porodica JOIN-ova

Ako vas pitaju za širu sliku, dobro je pokazati da „LEFT” i „INNER” nisu jedina dva:

- **RIGHT JOIN** — simetričan **LEFT**-u; čuva sve retke *desne* tablice. $A \text{ RIGHT JOIN } B \equiv B \text{ LEFT JOIN } A$.
- **FULL OUTER JOIN** — čuva retke bez para s *obje* strane.
- **CROSS JOIN** — Kartezijev produkt (svaki-sa-svakim), bez uvjeta.

Koristan trik: **LEFT JOIN ... WHERE desna.kljuc IS NULL** je *anti-join* — vraća baš one lijeve retke koji *nemaju* para (npr. „studenti koji nisu upisali nijedan kolegij”).

Zamka / caveat: NULL u uvjetu spajanja

NULL nije jednak ničemu, pa ni drugom NULL-u ($\text{NULL} = \text{NULL} \rightarrow \text{nepoznato}$). Zato retci s NULL vrijednošću u stupcu spajanja nikad ne dobiju para — ni u **INNER** ni u **OUTER JOIN**-u tamo gdje se traži jednakost. Ista logika stoji iza toga zašto **UNIQUE** dopušta više NULL-ova (vidi poglavlje o ograničenjima).

1.2 Normalizacija

Definicija: normalizirana baza

[ispitno pitanje]

Kad kažemo da je baza „normalizirana”, obično mislimo da je u **trećoj normalnoj formi (3NF)**. 3NF podrazumijeva da je baza u 2NF i da u tablicama **ne postoje tranzitivne ovisnosti** (neključni atribut ne smije ovisiti o drugom neključnom atributu).

Primjer: tranzitivna ovisnost

Neka tablica ima attribute (OIB, poštanski_broj, naziv_mjesta). Poštanski broj funkcijski ovisi o OIB-u, ali *naziv mjesta ovisi o poštanskom broju*, a ne izravno o OIB-u — to je tranzitivna ovisnost $OIB \rightarrow p.broj \rightarrow mjesto$. Rješenje: naziv mjesta i poštanski broj sele se u zasebnu tablicu *mjesto*, povezanu stranim ključem.

Za usmeni: ljestvica normalnih formi

Profesor traži samo 3NF, ali na usmenom je jako dobro pokazati da znate „ljestve” — svaka forma pretpostavlja prethodnu:

- **1NF** — svi atributi su *atomarni* (nema listi/ugniježđenih vrijednosti u ćeliji) i postoji ključ.
- **2NF** — 1NF *plus* nema *parcijalnih* ovisnosti: nijedan neključni atribut ne ovisi samo o *dijelu* složenog ključa.
- **3NF** — 2NF *plus* nema *tranzitivnih* ovisnosti (neključni $\rightarrow$ neključni).
- **BCNF** — stroža 3NF: *svaki* determinant mora biti kandidat-ključ.

Mnemotehnika (šaljivi klasik): „ključ, cijeli ključ i ništa osim ključa” — 1NF traži da atributi ovise o ključu, 2NF o *cijelom* ključu, 3NF *ni o čemu osim ključa*.

Zamka / caveat: normalizacija nije uvijek cilj

Ne recite da je „što normaliziranje to bolje”. Normalizacija uklanja redundanciju i anomalije (insert/update/delete), ali povećava broj JOIN-ova. U analitičkim/OLAP sustavima i kod NoSQL agregatnih modela često se namjerno **denormalizira** radi performansi. To je dobar most prema kasnijim temama (agregatni model, polyglot persistence).

2. Pravila integriteta i indeksi

2.1 Referencijski integritet i ograničenja

Definicija: referencijski integritet

[ispitno pitanje]

Odnosi se na održavanje **ispravnih veza između tablica**: vrijednost stranog ključa mora uvijek pokazivati na *postojeći* zapis u referenciranoj tablici (ili biti NULL). Održava se primarnim i stranim ključevima te ograničenjima NOT NULL, CHECK i UNIQUE.

Autorefleksija (self-reference). [ispitno pitanje] Tablica ima strani ključ koji referencira *samu sebe*. Služi za modeliranje **hijerarhija** — npr. zaposlenik(id, ..., voditelj_id) gdje voditelj_id pokazuje na zaposlenik.id. Time se u jednoj tablici prikazuje odnos nadređeni–podređeni.

ON DELETE (i ON UPDATE). [ispitno pitanje] Definira što se događa s *referencirajućim* zapisima kad se obriše (ili izmijeni) zapis u roditeljskoj tablici:

- CASCADE — brišu se (ili se ažuriraju) i referencirajući zapisi.
- RESTRICT — transakcija se odmah zaustavlja s greškom.
- NO ACTION — provjera se odgađa za kraj naredbe/transakcije (*default*); u praksi slično RESTRICT-u, ali kasnije.
- SET NULL — strani ključ postaje NULL.
- SET DEFAULT — strani ključ poprima zadanu vrijednost.

Definicija: CHECK i UNIQUE

[ispitno pitanje]

CHECK osigurava da uneseni podaci zadovoljavaju logički (true/false) uvjet; navodi se u definiciji tablice (npr. CHECK (dob >= 18)). **UNIQUE** osigurava da se ista vrijednost ne pojavi više puta u stupcu (ili kombinaciji stupaca).

UNIQUE i NULL. [ispitno pitanje] UNIQUE zabranjuje ponavljanje istih *poznatih* vrijednosti, ali **dopušta više NULL vrijednosti**, jer se u SQL-u dva NULL-a smatraju *različitima* (NULL = „nepoznato”, a dvije nepoznanice nisu nužno jednake).

Definicija: UNIQUE vs. primarni ključ

[ispitno pitanje]

Oba jamče jedinstvenost, ali:

- **Primarni ključ:** točno jedan po tablici, *ne smije* biti NULL, i *identificira* svaki redak.
- **UNIQUE:** može ih biti više, *smije* sadržavati NULL, i osigurava jedinstvenost, ali ne mora identificirati redak.

Npr. JMBAG identificira studenta (kandidat za PK), dok je e-mail jedinstven (UNIQUE), ali ga ne koristimo kao identifikator.

2.2 Indeksi

Definicija: indeksi

[ispitno pitanje]

Indeksi su podatkovne strukture koje **ubrzavaju dohvat, sortiranje, grupiranje i spajanje** podataka — koriste se nad stupcima po kojima se često pretražuje ili sortira.

Čemu ne služe. [ispitno pitanje] Indeksi *ne* ubrzavaju INSERT/UPDATE/DELETE — štoviše, mogu ih usporiti, jer se pri svakoj izmjeni podataka mora ažurirati i indeks. Također zauzimaju **značajan prostor**. Zato ih treba izbjegavati na malim tablicama i na stupcima koji se rijetko pretražuju, a često mijenjaju.

Za usmeni: kad indeks NE pomaže

Dobre dodatne točke za usmeni:

- **Niska kardinalnost** (npr. stupac „spol” s dvije vrijednosti) — indeks slabo selektira, optimizator ga često i ignorira.
- **Složeni (kompozitni) indeks** poštuje pravilo *leftmost prefix*: indeks (A, B) pomaže upitima po A i po (A, B), ali *ne* po samom B.
- **Clustered vs. non-clustered**: clustered indeks određuje *fizički* redoslijed redaka (jedan po

tablici), non-clustered je zasebna struktura s pokazivačima.

- **Covering index** — ako indeks sadrži sve stupce koje upit treba, baza uopće ne mora dohvaćati redak iz tablice.

2.3 B-stablo

Definicija: B-stablo

[ispitno pitanje]

Balansirano stablo koje se koristi za izgradnju indeksa. Pri dodavanju i brisanju podataka čvorovi se *cijepaju i spajaju* tako da stablo ostane uravnoteženo — cilj je da se do bilo kojeg zapisa dođe u *približno jednakom* (logaritamskom) broju koraka.

Zašto baš B-stablo? [ispitno pitanje] Jer omogućuje vrlo brzo pretraživanje uz mali broj pristupa u odnosu na sekvencijalno pregledavanje cijele tablice (pretraživanje stabla je reda $O(\log n)$, sekvencijalno $O(n)$). U pravilu su **listovi** B-stabla pokazivači na stvarne zapise.

Za usmeni: B-stablo vs. B⁺-stablo

Ako spomenete da „listovi pokazuju na zapise”, zapravo opisujete **B⁺-stablo** — varijantu koju stvarno koriste gotovo svi RDBMS-ovi (PostgreSQL, MySQL/InnoDB, Oracle):

- svi podaci/pokazivači su u *listovima*, unutarnji čvorovi drže samo usmjeravajuće ključeve $\Rightarrow$ viši fan-out, plići cache-i;
- listovi su *ulančani* u sortiranu listu $\Rightarrow$ vrlo brz **range scan** (BETWEEN, ORDER BY).

Zašto je balansirano bitna: jamči da je *najgori* slučaj i dalje $O(\log n)$; nebalansirano binarno stablo može degenerirati u listu ($O(n)$). Alternativa za jednakost (ne za raspone) je **hash indeks**.

3. Programabilnost baze: funkcije, procedure i okidači

Zajednička ideja cijelog poglavlja: dio poslovne logike se **sprema unutar baze**, a ne u aplikaciji. Umjesto da aplikacija svaki put šalje mnogo pojedinačnih SQL naredbi, baza već ima spremljen program koji se izvrši na poziv. To smanjuje mrežni promet i rasterećuje klijenta.

3.1 Funkcije vs. pohranjene procedure

Definicija: funkcija i procedura

[ispitno pitanje]

Funkcija je pohranjeni objekt koji (opcionalno) prima parametre i *uvijek vraća vrijednost* (ili tablicu); može se koristiti *unutar* SQL izraza (SELECT, WHERE, ORDER BY). **Pohranjena procedura** je skup SQL naredbi koji izvršava određeni zadatak, *smije mijenjati podatke i ne mora vraćati* vrijednost; poziva se zasebno (CALL), ne unutar izraza.

Ključna razlika: funkcija *vraća* vrijednost i živi u izrazima; procedura *obavlja* složene operacije nad bazom. Performanse su općenito vrlo dobre jer se izvršavaju na poslužitelju (manje mrežnog prometa, rasterećen klijent i backend).

Veza sa sigurnošću. [ispitno pitanje] Procedure povećavaju sigurnost jer korisnicima daju pravo izvršavanja *unaprijed definiranih* operacija bez *izravnog* pristupa tablicama. Primjer: student ne može sam pisati u tablicu prijava ispita, ali smije pozvati proceduru koja provjeri uvjete pa upiše samo ono što je dopušteno. Time se kontrolira *što* korisnik može, a sprječava neovlašteno mijenjanje.

3.2 DDL nad procedurama: CREATE / ALTER / DROP

Definicija: CREATE, ALTER, DROP

[ispitno pitanje]

CREATE stvara *novi* objekt (kad još ne postoji). **ALTER** *mijenja postojeći* objekt (npr. definiciju procedure ili stupce tablice) — objekt ostaje, mijenja se sadržaj/struktura. **DROP** *trajno briše* objekt — nakon toga ga treba iznova stvoriti.

Za usmeni: zašto CREATE OR REPLACE

U praksi (PostgreSQL) izmjena funkcije radi se s `CREATE OR REPLACE FUNCTION`, čime se izbjegava `DROP+CREATE` koji bi srušio ovisne objekte (npr. prava/grantove ili druge objekte koji je referenciraju). Poanta za usmeni: **ALTER** „čuva” objekt i njegove ovisnosti, **DROP** ih uništava.

3.3 Kontrola toka i višestruki rezultati

CASE. [ispitno pitanje] Uvjetna logika u SQL-u — bira različite vrijednosti ili radnje ovisno o uvjetu. SQL ekvivalent naredbe `IF-ELSE` iz programskih jezika.

Obrada više rezultata SELECT-a. [ispitno pitanje] Rezultati se obrađuju **FOR petljom** koja prolazi kroz svaki redak; podaci se spremaju u više pojedinačnih varijabli ili u jednu varijablu tipa `RECORD`.

Definicija: RETURN QUERY vs. RETURN NEXT

[ispitno pitanje]

Koriste se u funkcijama koje vraćaju više redaka. Za razliku od običnog `RETURN`, **ne prekidaju** izvršavanje, nego *dodaju* retke u izlazni skup pa funkcija nastavlja rad:

- `RETURN NEXT` — vraća *jedan* redak odjednom (tipično unutar petlje).
- `RETURN QUERY` — vraća *sve* retke jednog `SELECT` upita odjednom.

3.4 Iznimke (exceptions)

Definicija: iznimke

[ispitno pitanje]

Pogreške ili neočekivane situacije tijekom izvršavanja funkcije, procedure ili triggera (dijeljenje s nulom, nepostojeći podaci, pokušaj umetanja duplikata itd.). U SQL-u se hvataju na **razini bloka** i obrađuju na kraju bloka (`EXCEPTION WHEN ...`) kako program ne bi neočekivano prekinuo s greškom. Ručno se dižu s `RAISE EXCEPTION`.

3.5 Okidači (triggers)

Definicija: okidač

[ispitno pitanje]

Objekt u bazi koji se **automatski izvrši** kad se dogodi određeni događaj (`INSERT/UPDATE/DELETE`). Može provjeriti ispravnost, izmijeniti podatke, upisati u drugu tablicu, obrisati povezane zapise ili *sprečiti* naredbu bacanjem pogreške. Ukratko: osigurava dodatni integritet ili pokreće dodatne zadaće.

Struktura naredbe `CREATE TRIGGER` odgovara na pitanja: *što se dogodilo* (`INSERT/UPDATE/DELETE`), *kada* (`BEFORE/AFTER`), *koliko puta* (`FOR EACH ROW/STATEMENT`), *uz koje uvjete* (`WHEN`) i *što treba obaviti* (funkcija za okidač).

BEFORE vs. AFTER. [ispitno pitanje]

- **BEFORE** — izvršava se *prije* spremanja. Koristi se za *provjeru* i *izmjenu* vrijednosti prije upisa te za *sprječavanje* naredbe. *Primjer*: dopušteni minus -100 €; korisnik traži stanje -200 €; **BEFORE UPDATE** okine `RAISE EXCEPTION` i **UPDATE** se *ne* izvrši, stanje ostaje netaknuto.
- **AFTER** — izvršava se *nakon* uspješnog upisa. Koristi se za radnje koje *ne* mijenjaju samu promjenu: logiranje, obavijesti, ažuriranje drugih tablica. Budući da je zapis već spremljen, **AFTER** ga više ne može spriječiti.

ROW vs. STATEMENT. [ispitno pitanje]

- **FOR EACH ROW** — izvrši se *jednom po retku*; ima pristup vrijednostima `OLD` i `NEW` (tip `RECORD`). *Primjer*: `UPDATE student SET godina = godina+1` nad 50 studenata okine **ROW trigger** 50 puta.
- **FOR EACH STATEMENT** — izvrši se *jednom po naredbi*, bez obzira na broj redaka; *nema* smislen `OLD/NEW` po retku. Koristi se za logiranje operacija i dodatne provjere dozvola.

Primjer: kombinacije za UPDATE nad 100 redaka

BEFORE ROW	100 puta, prije promjene svakog retka
AFTER ROW	100 puta, nakon promjene svakog retka
BEFORE STATEMENT	1 put, prije cijele naredbe
AFTER STATEMENT	1 put, nakon cijele naredbe

Okidači u kaskadi. [ispitno pitanje] Jedan trigger može pokrenuti drugi ako njegova funkcija izvede `INSERT/UPDATE/DELETE` nad tablicom koja *također* ima trigger. *Primjer*: **AFTER INSERT** na narudžba smanji zalihi u skladiste; **AFTER UPDATE** na skladiste provjeri je li količina pala ispod minimuma i upiše zahtjev za nabavu. Opasnost: teško uočljive dodatne operacije, sporije izvršavanje i — u najgorem slučaju — **beskonačna petlja** (trigger ažurira tablicu koja ponovo aktivira isti trigger). Zato se funkcije, procedure i triggeri tretiraju kao *programski kod baze* i drže pod verzioniranjem (npr. Git).

Definicija: funkcija za okidač

[ispitno pitanje]

Funkcija koja sadrži logiku triggera i vraća vrijednost tipa `TRIGGER`; mora biti deklarirana s `RETURNS trigger`, a trigger je poziva s `EXECUTE FUNCTION`. Kod **BEFORE** triggera povratna vrijednost je *važna*: `RETURN NEW/OLD` propušta izmjenu, a `RETURN NULL` *preskače* daljnju operaciju nad tim retkom. Kod **AFTER** triggera povratna vrijednost se *ignorira*.

Za usmeni: OLD/NEW po operaciji i INSTEAD OF

Sitnica koja ostavlja dobar dojam:

- Kod `INSERT` postoji samo `NEW`; kod `DELETE` samo `OLD`; kod `UPDATE` oba.
- **INSTEAD OF** trigger postoji za *pogled* (`VIEW`) — njime se „ne-ažurni” pogled može učiniti zapisljivim tako da preusmjeri izmjenu na osnovne tablice.

4. Napredne teme relacijskih baza

4.1 Složeni tipovi i polustrukturirani podaci

Složeni tipovi. **[ispitno pitanje]** ARRAY (polje elemenata iste vrste, fiksne veličine), VARRAY (polje promjenjive veličine, ali s unaprijed određenim maksimumom), ROW (zapis od više različitih atributa), SET (skup jedinstvenih elemenata, redosljed nevažan), LIST (kod ovog kolegija: jedinstveni elementi, redosljed važan).

Zamka / caveat: definicija LIST-a

Kod ovog kolegija LIST je definiran kao „jedinstveni elementi, redosljed važan”. U općoj informatici LIST obično *dopušta duplikate* (za razliku od SET-a), a razlikovnica prema SET-u je upravo redosljed. Na usmenom slijedite definiciju s predavanja, ali budite svjesni te razlike ako vas potpitaju.

XML i JSON. **[ispitno pitanje]** Obično se pohranjuju *kao tekst*, ali baza nudi posebne operatore i funkcije za rad s njima (npr. PostgreSQL `jsonb` s indeksiranjem i operatorima poput `->`, `->`, `@>`).

GIS. **[ispitno pitanje]** **Geografski informacijski sustav** — sustav za prikupljanje, pohranu, analizu i prikaz *prostornih* podataka. Osnovne komponente: točke, linije, poligoni (koordinate, udaljenosti, granice). Bitno: GIS služi za *analizu prostornih odnosa*, ne samo za crtanje karata. Prostorni podaci pohranjuju se posebnim tipovima (geometrija, geografija, raster) uz prostorna proširenja poput **PostGIS**-a ili Oracle Spatiala.

4.2 Vremenski i bitemporalni podaci

Vremenski (temporalni) podaci. **[ispitno pitanje]** Služe za pohranu *povijesti* — praćenje *od kada do kada* neki podatak vrijedi. Npr. osoba je živjela na adresi A od 1.1.2000. do 1.7.2014., a zatim na adresi B od 2.7.2014. do danas.

Problem primarnog ključa. **[ispitno pitanje]** Jedan objekt ima *više* zapisa kroz vrijeme (osoba s ID 1124 ima više adresa u različitim razdobljima). Rješenje: **kompozitni primarni ključ** (`sifra`, `datumOd`). Dodatno, **trigger** automatski zatvara prethodni zapis (postavi `datumDo`) kako se razdoblja ne bi preklapala.

Definicija: bitemporalne baze

[ispitno pitanje]

Baze koje unutar relacija koriste *obje* vremenske dimenzije: **vrijeme valjanosti** (valid-time / aplikativno vrijeme) i **sistemske vrijeme** (system-time / transakcijsko vrijeme) — za razliku od *temporalnih*, koje koriste samo jednu.

- **Aplikativno (valid) vrijeme** — kada je podatak vrijedio *u stvarnosti* (npr. osoba je na adresi živjela 1.1.2000.–1.7.2014.).
- **Sistemske (transakcijsko) vrijeme** — kada je zapis bio valjan *u samoj bazi*: kada je transakcijom ubačen, promijenjen ili izbrisan (npr. ubačen 3.2.2000. 10:34, izmijenjen/izbrisan 1.9.2014. 18:12).

Za usmeni: mentalni model 2×2

Zamislite dvije nezavisne osi vremena. Temporalna baza prati jednu os, bitemporalna obje. Poanta koju profesori vole čuti: *sistemske vrijeme nije vrijeme kada je podatak vrijedio u stvarnosti* — to je čest izvor zabune. Standard SQL:2011 uveo je nativnu podršku (`PERIOD FOR`, `FOR SYSTEM_TIME AS OF ...`).

4.3 Distribuirane vs. centralizirane baze

Definicija: distribuirana baza

[ispitno pitanje]

Baza u kojoj se podaci nalaze na *više lokacija* (fizičkih ili logičkih), pri čemu na svakom poslužitelju postoji vlastiti SUBP koji komunicira s ostalima. Podaci su razdvojeni po poslužiteljima; dio (zajednički podaci, šifarnici) može se *preslikati*; shema je uglavnom ista, a konkretni podaci u tablicama različiti.

	Centralizirana	Distribuirana
Prednosti	jednostavno održavanje; jedno mjesto za HW/SW; jeftinije licence	manje mrežno opterećenje; manje opterećenje baza; jeftiniji HW; neovisnost o ostatku
Rizik	jedna točka kvara; usko grlo	složenost, sinkronizacija, konzistentnost

Koja je bolja? **[ispitno pitanje]** Nema univerzalnog odgovora — ovisi o poslovnom kontekstu:

- **Konzum** — vjerojatno *centralizirani* pristup: mnogo geografski raspršenih poslovnica, ali želimo jedinstvenu, visoko dostupnu sliku (ne želimo da poslovnica stane jer je pala mreža u Zagrebu) + lokalni pristup.
- **Lanac autosalona** — vjerojatno *distribuirani*: manje salona, nisu jako raspršeni, a želi se ista, ažurna evidencija vozila.

4.4 Transakcije i 2PC

Definicija: transakcija

[ispitno pitanje]

Niz naredbi proglašen **nedjeljivom, jednom logičkom cjelinom** koji se mora izvršiti *u cijelosti* ili se *uopće* ne smije izvršiti („sve ili ništa”).

Definicija: dvofazni protokol (2PC)

[ispitno pitanje]

2PC (two-phase commit) osigurava uspješnu provedbu *distribuirane* transakcije (na više poslužitelja/čvorova). Provodi se u dvije faze:

1. **Faza pripreme** — koordinator obavijesti sve sudionike; svaki odgovara PRE-COMMIT (spreman) ili odustaje. Ako *barem jedan* odustane, koordinator šalje ROLLBACK svima.
2. **Faza commita** — koordinator šalje COMMIT; sudionici commitaju. Ako neki ne uspije, koordinator šalje abort svima → ROLLBACK na svima.

Zamka / caveat: blokiranje kod 2PC

Glavna slabost 2PC-a: *loše se ponaša pri mrežnim problemima*. Ako koordinator otkáže usred commit-faze, sudionici mogu ostati **zaglavljani** (blocking) čekajući odluku, držeći lokote. Zato postoji **3PC** (trofazni), koji uvodi dodatnu „pre-commit” fazu da izbjegne blokiranje — dobra dodatna rečenica ako vas pitaju „kako se to rješava”. Moderni sustavi (npr. Google Spanner) koriste konsenzus-protokole poput *Paxos/Raft*.

4.5 Visoka dostupnost, klasteriranje, failover

Definicija: HA, redundancija, klasteriranje

[ispitno pitanje]

Visoka dostupnost (HA) znači da je baza dostupna gotovo neprekidno, i uz kvar HW/SW/mreže; cilj je minimizirati *downtime*. **Redundancija** (rezervne komponente) osnovni je način postizanja HA. **Klasteriranje** povezuje više poslužitelja u jedan sustav radi pouzdanosti i performansi: jedan je *primarni*, ostali *sekundarni* (mogu dijeliti diskove).

Konfiguracije klastera: **active-passive** (sekundarni samo slušaju izmjene na primarnom) i **active-active** (sekundarni također primaju konekcije i poslužuju korisnike).

Failover. [ispitno pitanje] Sposobnost SUBP-a da prepozna *havariju* primarnog poslužitelja, proglasi sekundarni novim primarnim i preusmjeri sav promet na njega.

Za usmeni: RTO i RPO

Dvije brojke koje profesionalci uvijek spominju uz HA/backup: **RTO** (Recovery Time Objective) — koliko dugo smijemo biti u prekidu; **RPO** (Recovery Point Objective) — koliko podataka smijemo izgubiti (mjeri se „unatrag” od trenutka kvara). Češći backup → manji RPO.

4.6 Skalabilnost i backup

Definicija: vertikalna vs. horizontalna skalabilnost

[ispitno pitanje]

Horizontalna = „više strojeva” — dodavanje novih poslužitelja/ uređaja/VM-ova u postojeću mrežu (*scale-out*). **Vertikalna** = „jači stroj” — povećanje resursa *postojećeg* poslužitelja (*scale-up*).

Backup. [ispitno pitanje] Izrada pričuvne kopije baze. Nije primarna, ali jest mjera osiguranja veće dostupnosti (povrat iz kopije jedan je od načina uspostave sekundarnog poslužitelja). Backupirati treba *sve* podatke SUBP-a i logičke (žurnal) dnevnike u koje se upisuju transakcije. Svaki produkcijski sustav mora imati redoviti backup — najmanje jednom dnevno, uz backup logičkih dnevnika po završetku svakog backupa.

4.7 Zašto relacijske baze

Zašto RDBMS. [ispitno pitanje] Dobre su za većinu poslovnih potreba, imaju jednostavna pravila, sve koriste (jednostavan) SQL, svi programski jezici imaju sučelja prema njima, a sami sustavi postoje jako dugo (zreli, provjereni).

Kako odabrati konkretan RDBMS. [ispitno pitanje] Ovisi o tome što nam treba, o cijeni/vrijednosti, sklonosti open-source rješenjima, dostupnosti programera za tu tehnologiju i slično.

5. NoSQL: temelji, vrste i modeli

5.1 Što je NoSQL i koje su karakteristike važne

Definicija: NoSQL

[ispitno pitanje]

Sljedeća generacija SUBP-ova koji *uglavnom nisu relacijski*, tipično su **distribuirani**, **otvorenog koda** i **horizontalno skalabilni**.

Važne karakteristike NoSQL-a: **[ispitno pitanje]**

- **schemaless** (nemaju eksplicitnu shemu podataka);
- jednostavna **replikacija**;
- jednostavno korištenje putem **API-ja** (za razliku od relacijskih, gdje aplikacija koristi *drivere* koji uspostavljaju vezu, šalju SQL i prevode rezultat);
- **nisu (nužno) konzistentne** — BASE umjesto ACID;
- rade s **velikim količinama podataka**;
- bolje u *horizontalnoj* skalabilnosti (SQL baze su bolje u vertikalnoj — logično, SQL je nastao u doba bez Interneta).

Zamka / caveat: NoSQL $\neq$ „No SQL“

Naziv zavarava: mnoge NoSQL baze imaju upitne jezike (Cypher, Mongo query language) i *nije* riječ o odsustvu upita, nego o *nerelacijskom* modelu. Uobičajeno se čita kao „Not only SQL“.

5.2 Vrste NoSQL baza

Definicija: četiri vrste

[ispitno pitanje]

Dijele se prema *logičkom* modelu podataka (ne fizičkom smještaju).

Vrsta	Model / primjena	Primjeri
key-value	ključ = jedinstveni identifikator, vrijednost = bilo što; operacije unos/izmjena/dohvat/brisanje po ključu; npr. profilne slike (ključ = username)	Dynamo, Voldemort, Riak, Redis
dokumentske	ključu je pridružen <i>dokument</i> (strojno čitljiv: XML, JSON); pretraživanje i po sadržaju dokumenta	MongoDB
column-family	dvorazinska mapa (mapa mapa): ključ 1. razine = redak, 2. razine = stupac; stupci u familijama; big data	Apache (Cassandra/HBase)
grafovske	vrhovi i bridovi (bridovi mogu biti usmjereni); pristup API-jem ili Cypherom	Neo4j, GraphDB

5.3 Konzistentnost: relacijske vs. nerelacijske

U **relacijskim bazama** **[ispitno pitanje]** konzistentnost znači da transakcija prevodi podatke iz jednog *konzistentnog* stanja u drugo, poštujući pravila baze (strani ključevi, ograničenja). Kod distribuiranih transakcija dodatno treba potvrda *svih* poslužitelja („sve ili nigdje“).

U **NoSQL bazama** **[ispitno pitanje]** postoji **eventualna konzistentnost**: svi poslužitelji *s vremenom* završe u istom stanju, ali ne nužno odmah nakon zapisa (npr. prag od 50% znači da će promjena

odmah zahvatiti pola poslužitelja, a ostali će je „dostići” kasnije).

5.4 Agregatni model

Definicija: agregatni model

[ispitno pitanje]

Model u kojem se **agregat** — skupina povezanih podataka (dokument iz dokumentskih baza, redak/mapa iz CF baza) — promatra kao *osnovna jedinica* dohvaćanja/spremanja, sa svojim jedinstvenim identifikatorom. Primjeri agregata: upisni list studenta, cijeli račun, košarica u web-trgovini.

Prednost: bolje performanse — povezani podaci su fizički zajedno, pa se smanjuje potreba za JOIN-ovima nad više skupova. U relacijskom modelu isti su podaci razbijeni po više tablica (odatle česti JOIN-ovi).

Zamka / caveat: grafovske baze NISU agregatni model

Iako su NoSQL, **grafovske baze ne pripadaju agregatnom modelu** — kod njih podaci nisu grupirani u aggregate, nego organizirani kao čvorovi i bridovi (druga granulacija). Ovo je klasično potpitanje.

5.5 Shema podataka i schemaless

Definicija: shema podataka

[ispitno pitanje]

Detaljan opis smještaja svih podataka domene u bazi. U relacijskoj bazi to znači: predefinirati entitete i attribute, shvatiti domene i funkcijske zavisnosti, rasporediti attribute u tablice, normalizirati.

NoSQL baze su **schemaless** — nije eksplicitno navedeno što se i kako pohranjuje. Posljedice:

- **Prednost — fleksibilnost za developere.** U relacijskom modelu aplikacija poznaje shemu; svaku promjenu treba uskladiti i u bazi i u aplikaciji (uz downtime ili nezgrapne workaroundove) — traži sinkronizaciju baze i aplikacije. U NoSQL-u aplikacija odmah koristi promijenjenu shemu; migracija je manje mučna.
- **Nedostatak — izostanak sheme.** Baza bi bolje optimizirala dohvat/izmjenu kad bi poznavala shemu; nemoguća je validacija (npr. imaju li svi atributi godina isti tip: '2001' vs. 2001).

Važno: i u schemaless bazama postoji *implicitna* shema koju pretpostavljaju aplikacije; developeri moraju održavati internu dokumentaciju s konvencijama imenovanja.

5.6 Zašto NoSQL i za što

Kada NoSQL. [ispitno pitanje] Kada operacije nad podacima pokazuju *bolje performanse* nego u relacijskom modelu — ali samo za ono za što je ta baza namijenjena. Primjer: grafovsku bazu koristimo kad nas zanimaju sličnosti i „dublje veze” u ogromnim skupovima (npr. traženje drugog, trećeg kruga prijatelja na društvenoj mreži), ali *ne* za agregiranje.

5.7 CAP teorem

Definicija: CAP teorem

[ispitno pitanje]

U distribuiranom sustavu baza moguće je istovremeno postići samo **dvoje** od: **konzistentnost (C)**, **dostupnost (A)**, **otpornost na particiju mreže (P)**.

- **Dostupnost** — svaki upit funkcionalnom poslužitelju dobiva odgovor.
- **Konzistentnost** — svaki odgovor poslan korisniku je točan (konzistentnost cijelog sustava).
- **Partition tolerance** — kad se zbog mrežnog kvara pojave odvojene skupine poslužitelja, sustav i dalje radi.

Što CAP znači u praksi. [ispitno pitanje] Izbor nije binaran ni šablonski — ovisi o bazi i poslovnom kontekstu. No o jednome nema rasprave: **P je obavezan** u modernim distribuiranim sustavima. Zato izbor pada na *C* ili *A*. NoSQL baze mogu žrtvovati konzistentnost za dostupnost, uz rizik problema *pomirivanja* podataka (usklađivanje verzija nastalih na različitim poslužiteljima — kupac kupi kod poslužitelja *A*, ali se zbog prekida mreže ne zapiše na *B*). *Primjer gdje C pobjeđuje A*: bankovni sustavi / internet bankarstvo — važnije je da stanje računa bude točno nego da sustav uvijek odgovori.

Za usmeni: CP / AP / CA i PACELC

Klasifikacija sustava po tome koja dva svojstva biraju:

- **CP** (žrtvuju *A* pri particiji): npr. MongoDB, HBase.
- **AP** (žrtvuju *C* pri particiji): npr. Cassandra, Dynamo, Riak.
- **CA** (bez particija) — realno samo klasične jednočvorne RDBMS; u *distribuiranom* svijetu *CA* nije održiv jer particije postoje.

Moderna dopuna je **PACELC**: ako particija (*P*), biraj *A* ili *C*; *else* (*E*), biraj između latencije (*L*) i konzistentnosti (*C*). Time se naglašava da postoji trade-off i *bez* kvara mreže.

5.8 ACID vs. BASE

Definicija: ACID

[ispitno pitanje]

Svojstva koja svaka transakcija u relacijskoj bazi mora zadovoljiti:

- **Atomicity** (nedjeljivost) — sve ili ništa.
- **Consistency** (konzistentnost) — iz jednog konzistentnog stanja u drugo, poštujući pravila baze.
- **Isolation** (izolacija) — paralelne transakcije daju rezultat kao da su tekle *sekvencijalno*; postiže se lokotima i 2PC zaključavanjem.
- **Durability** (trajnost) — učinci uspješne transakcije preživljavaju i kvar sustava neposredno nakon commita.

Definicija: BASE

[ispitno pitanje]

Umjetna kratica skovana kao kontrast ACID-u:

- **Basically Available** — sustav je uvijek dostupan.
- **Soft-state** — stanje se može mijenjati i bez korisničke akcije.

- **Eventually consistent** — s vremenom postaje potpuno konzistentan (ako nema novih akcija); dok se to ne dogodi, čitanja su moguća, makar ne odražavala najnovije stanje.

U suštini: **ACID osigurava konzistentan**, a **BASE uvijek dostupan** sustav; BASE je „u duhu” CAP teorema (bira A na štetu trenutne C). Zgodna dosjetka: ACID = „kiselo/strogo”, BASE = „bazično/popustljivo” — kemijska metafora nije slučajna.

5.9 Key-value i MongoDB (detaljnije)

Key-value. **[ispitno pitanje]** Najjednostavniji oblik NoSQL-a: ključ je jedinstveni identifikator, vrijednost može biti jednostavna (tekst, datum, broj) ili složena (polje, struktura, binarni objekt). Prednosti: odlična i jeftina horizontalna skalabilnost, izvrsne performanse kod čestih čitanja/ pisanja uz malo ažuriranja, jednostavna shema (nema stranih ključeva ni pretraživanja po vrijednosti). Primjene: korisnički podaci na webu, konfiguracijski parametri, *cache*, *session storage*, oglasi, blockchain, multimedija. Primjer: Riak.

MongoDB. **[ispitno pitanje]** Primjer *dokumentske* baze. Podaci se pohranjuju u **kolekcije** JSON dokumenata (kolekcija je ekvivalent tablice u relacijskoj bazi). Pristup preko API-ja, uz podršku za mnogo programskih jezika.

6. Grafovske baze i Cypher

Definicija: property graph

[ispitno pitanje]

Klasični model usmjerenog/neusmjerenog grafa s **vrhovima** i **bridovima**. Vrhovi i bridovi mogu imati **oznake (labels)**, tj. tipove (npr. Osoba, Student za vrh; SLUŠA, PREDAJE za brid), te **svojstva (properties)** — parove ime–vrijednost. Bridovi i vrhovi istog tipa *ne* moraju imati ista svojstva (schemaless). Primjer: Neo4j (trajno perzistiranje, transakcije — što je rjeđe u NoSQL svijetu).

Ključna riječ WITH u Cypheru. **[ispitno pitanje]** Služi za *ulančavanje* upita — koristi se umjesto RETURN da rezultate prethodnog dijela prenese kao *variable* u sljedeći. Sve što se prenosi mora biti *imenovano* nakon WITH.

```
// Osobe starije od prosjeka: WITH prenosi prosjek u sljedeći MATCH
MATCH (p1:Person)
WITH AVG(p1.age) AS prosjek
MATCH (p2:Person) WHERE p2.age > prosjek
RETURN p2;
```

Za usmeni: zašto graf, a ne JOIN-ovi

Snaga grafovskih baza je u *prolasku kroz veze* (traversal). Upit „tko je u 3. krugu prijatelja” u relacijskoj bazi traži tri (ili više) skupih self-JOIN-ova; u grafu je to prirodan obilazak susjeda konstantne cijene po koraku. Zato su izvrsne za društvene mreže, preporuke i otkrivanje prijevara — ali *loše* za masovnu agregaciju (za to su bolje dokumentske/CF baze).

7. Map/Reduce i Aggregation Pipeline Framework

7.1 Ideja i faze

Definicija: Map/Reduce

[ispitno pitanje]

Programski **uzorak (pattern)** za izvođenje *distribuiranih* upita (nekad ga sami implementiramo, nekad je ugrađen). Patentirao ga Google 2004.; najširu primjenu ima u *agregatnim modelima* i *big data* obradi. Sastoji se od tri faze: **mapping** → **shuffle** → **reduction**.

Temeljna ideja: ulaz se dijeli na manje jedinice (u NoSQL bazama to je već napravljeno), grade se mape vrijednosti (*mapping*), potom se preorganiziraju (*shuffle* — reorganizacija u druge mape radi lakšeg računanja) i reduciraju (*reduction*) u konačne liste.

Veza s distribuiranim sustavima. [ispitno pitanje] NoSQL baze se vrte na klasterima od tisuću i više jeftinih, ne-high-end računala. Umjesto da se gigabajti podataka skupljaju na jedno mjesto, *šalje se kod* (kilobajti) na svaki čvor, izračun se provede lokalno, a natrag se prikupljaju samo *rezultati* (opet kilobajti). Time se drastično smanjuje mrežni promet. Načelo vrijedi za *bilo koji* klaster s mnogo čvorova (npr. Hadoop), ne samo NoSQL baze.

7.2 Primjer: analiza košarice

Primjer: analiza košarice

[ispitno pitanje]

Dani su računi (jedan račun = agregat); cilj je za svaki proizvod dobiti *prodanu količinu* i *prihod*. Kako cijena varira po poslovnica, za analizu (npr. prosjek) trebaju svi podaci.

- **Map** (na mapper čvorovima): ulaz je *jedan* agregat, izlaz su parovi ključ-vrijednost. Ovdje: ključ = naziv proizvoda, vrijednost = zapis o cijeni i količini. Budući da prima jedan agregat, mapiranje ide *paralelno*.
- **Shuffle**: nakon mapiranja, podaci s različitih čvorova objedinjuju se u nove mape (grupiranje po ključu).
- **Reduce** (na reducer čvorovima): ulaz je ključ, izlaz jedna (ili nijedna) vrijednost — listu vrijednosti za ključ svodimo na jednu (ovdje: ukupna količina i prihod po proizvodu).

7.3 Combiner, combinable reducer, finalize

Definicija: combiner / combinable reducer / finalize

[ispitno pitanje]

Combiner je „lokalni reducer” na *mapper* čvoru — odmah sumira istoimene proizvode prije slanja, čime smanjuje mrežni promet.

Combinable reducer je reduce funkcija napisana tako da može obraditi i *naknadno* pristigle (zakašnjele) podatke; framework je smije pozvati *više puta zaredom*. Zamjenjuje i combiner i reduce — ista funkcija radi na mapper i na reducer čvorovima (MongoDB *zahtijeva* da sve reduce funkcije budu combinable).

Finalize se poziva kad su *sva* mapiranja i reduciranja pouzdano gotova; u njoj se rade završna računanja.

Zamka / caveat: zašto prosjek NE smije u reducer

Combinable reducer smije nositi sume (cijena, količina), ali **ne prosjek** — jer se podaci mogu naknadno dopunjavati, pa bi međuprosjek bio besmislen. Formalno, combinable reducer

mora biti **asocijativan, komutativan i idempotentan** (ponovni poziv na već reduciranom rezultatu ne smije ga promijeniti). Prosjek nije asocijativan, pa se *prosjek računa u finalizacijskoj*, nakon MR-a. Ovo je klasično „ubodno” pitanje.

APF. [ispitno pitanje] Aggregation Pipeline Framework je alternativa Map/Reduce-u *unutar* MongoDB-a za distribuirane upite — suvremeniji, jednostavniji i potencijalno brži.

8. Polyglot persistence

Definicija: persistence i polyglot persistence

[ispitno pitanje]

Persistence je trajna pohrana podataka. **Polyglot persistence** je *hibridni* pristup: korištenje više *različitih* baza podataka za pohranu/korištenje/analizu u složenom sustavu, svaku za ono u čemu je najbolja.

Kako procijeniti dobar storage. [ispitno pitanje] Nema 100% točnog odgovora; treba razmisliti: s kakvim podacima baratamo, kakve vrijednosti postižu, koliko i kako treba osigurati integritet, i kako bismo implementirali pohranu u željenoj bazi.

Primjer: web-shop

[ispitno pitanje]

Sve podatke može držati jedna relacijska baza — ali polyglot pristup dodjeljuje svakoj vrsti podataka najprikladniju bazu:

- košarice i sesije → **key-value**;
- izvršene narudžbe → **dokumentske**;
- preporuke → **grafovske**;
- inventar i cjenik → **relacijske**.

Svaka aplikacija razgovara izravno s bazama koje koristi; korisno je *enkapsulirati* bazu u **service** s jedinstvenim sučeljem (npr. REST API) da se baze mogu razvijati neovisno o aplikacijama.

Prednosti i mane. [ispitno pitanje] Prednost: svaka vrsta podataka dobiva optimalnu tehnologiju. Mane:

- svaka baza treba barem jednog **administratora** (konfiguracija, održavanje, nadzor, backup);
- **složen razvoj** ako ne poznajemo tehnologije;
- **održavanje konzistentnosti** među bazama (ako je potrebno);
- općenito **složenost održavanja**.

Zlatno pravilo: koristiti samo tehnologije koje stvarno znamo održavati.

Za usmeni: most prema mikroservisima

Enkapsulacija baze iza servisa s API-jem točno je ono na čemu počiva *microservices* arhitektura („database per service”). Ako to spomenete, pokazujete da vidite širu sliku: polyglot persistence je prirodna posljedica razbijanja monolita na servise koji svaki biraju vlastitu pohranu.

9. NoSQL u RDBMS-ovima i NewSQL

Motivacija za novotarije. [ispitno pitanje] Ovisi o proizvođaču:

- **open-source** — proširenja pomažu developerima (praktičnost) i drže sustav ukorak s modernim tehnologijama (zadržavanje tržišnog udjela);
- **komercijalni** — zadržavanje klijenata (da ne bježe za novim tehnologijama, da ne padne broj licenci) i proširenje tržišta.

Praktična posljedica: moderni RDBMS-ovi danas imaju JSON/jsonb, prostorne tipove, itd. — granica prema NoSQL-u se briše.

Definicija: NewSQL

[ispitno pitanje]

Relacijske baze koje pokušavaju postići **skalabilnost NoSQL-a** za transakcijsko procesiranje, *zadržavajući* ACID svojstva i konzistentnost — „najbolje od oba svijeta”.

Za usmeni: primjeri NewSQL sustava

Konkretni predstavnici koje vrijedi imenovati: **Google Spanner, CockroachDB, VoltDB, TiDB**. Ideja: horizontalno skalabilan SQL uz jake garancije (Spanner koristi sinkronizirane satove i konsenzus za globalne ACID transakcije).

10. Brzo ponavljanje pred usmenim

Zgusnute suprotnosti koje profesori najčešće traže „iz cuga”.

Pojam	Jednom rečenicom
INNER vs LEFT	INNER = samo parovi; LEFT = svi lijevi + NULL za bespare; $LEFT \geq INNER$ redaka.
3NF	2NF + nema tranzitivnih ovisnosti (neključni $\nrightarrow$ neključni).
PK vs UNIQUE	PK: jedan, ne-NULL, identificira. UNIQUE: više, smije NULL, ne mora identificirati.
UNIQUE + NULL	Dopušta više NULL-ova (dva NULL-a su „različita”).
Indeks	Ubrzava čitanje/sort/join; usporava upis; troši prostor; ne za male tablice.
B ⁺ -stablo	Balansirano, listovi ulančani $\rightarrow$ brz range scan; $O(\log n)$.
Funkcija vs procedura	Funkcija vraća vrijednost i ide u izraze; procedura mijenja podatke, poziva se zasebno.
BEFORE vs AFTER	BEFORE: provjeri/izmijeni/spriječi; AFTER: log/obavijest, ne može spriječiti.
ROW vs STATEMENT	ROW: 1×/redak, ima OLD/NEW; STATEMENT: 1×/naredba.
Valid vs system time	Valid = kad vrijedi u stvarnosti; system = kad vrijedi u bazi.
2PC problem	Blokiranje ako koordinator padne u commit-fazi (rješenje: 3PC/konsenzus).
Vert. vs horiz.	Vertikalno = jači stroj; horizontalno = više strojeva.
CAP	Distribuirano: biraš 2 od C/A/P; P je obavezan $\Rightarrow$ biraš C ili A.
ACID vs BASE	ACID = konzistentnost; BASE = dostupnost + eventualna konzistentnost.
Agregatni model	Povezani podaci zajedno; graf NIJE agregatni.
MR faze	map $\rightarrow$ shuffle $\rightarrow$ reduce; prosjek tek u finalize.
Combinable reducer	Asocijativan, komutativan, idempotentan; nosi sume, ne prosjek.
Polyglot	Više baza, svaka za svoju svrhu; enkapsulacija iza servisa.
NewSQL	Skalabilnost NoSQL-a + ACID relacijskih baza.

Sretno na usmenom! Razumij vezu među temama — pola pitanja su zapravo mostovi (indeks $\rightarrow$ B⁺-stablo, transakcija $\rightarrow$ ACID $\rightarrow$ CAP $\rightarrow$ BASE, agregat $\rightarrow$ MR $\rightarrow$ polyglot).